

Computational Wind Engineering

Finite volumes II

Prof. Luca Patruno
University of Bologna

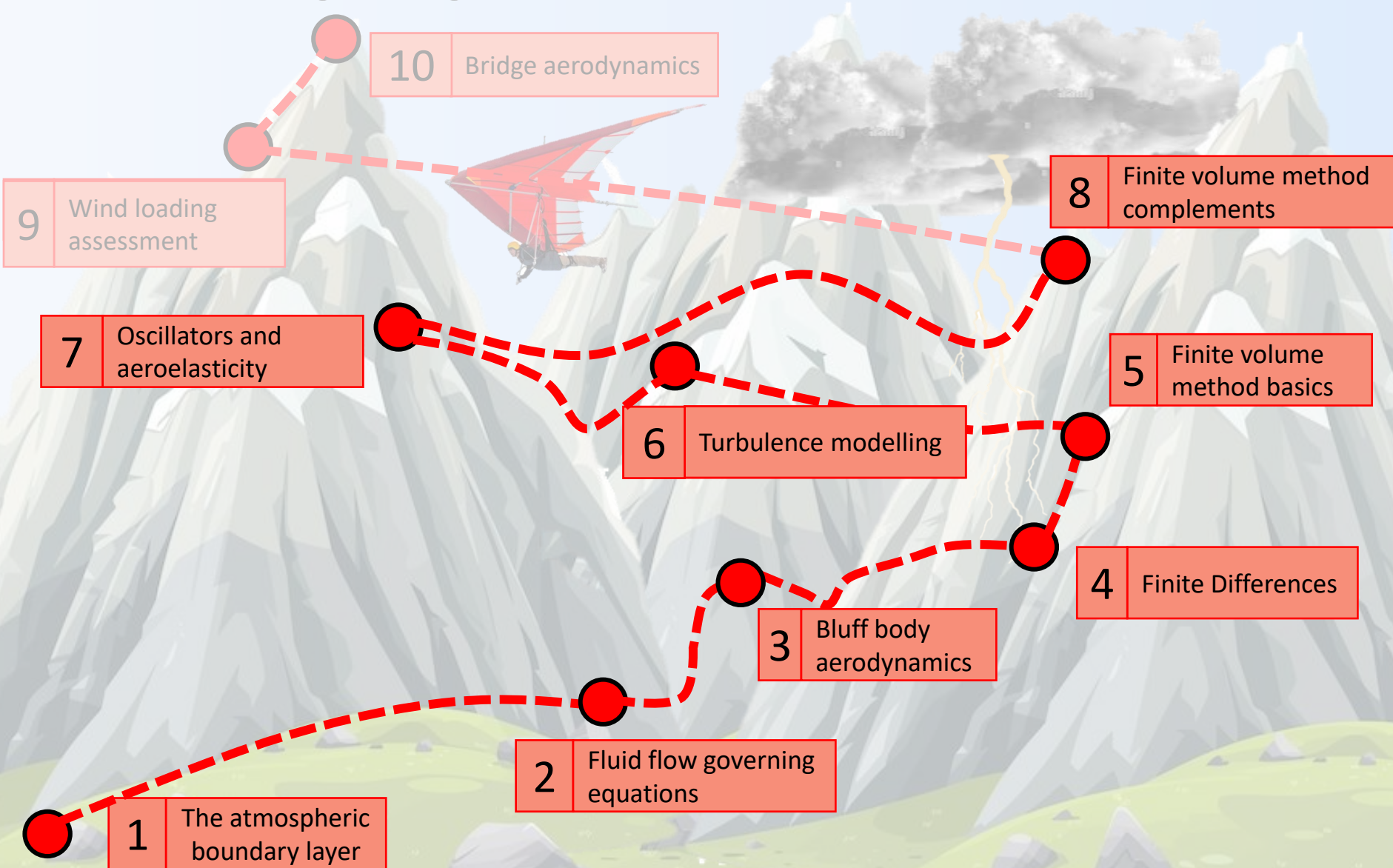
- Boundary conditions
- The wall BC
- Wall-functions
- TVD schemes
- Overview of the solution strategy

Map of the course

Wind Engineering

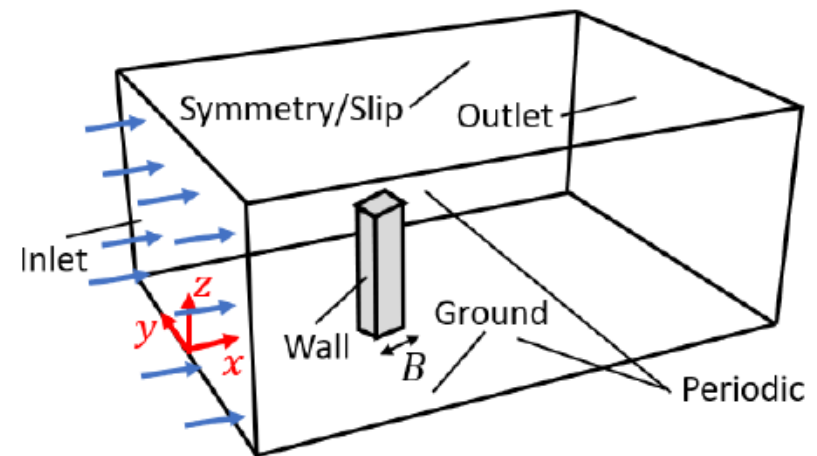
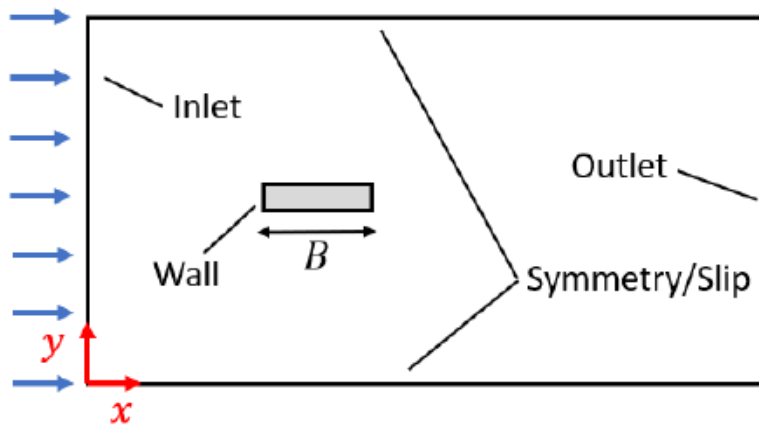
Fluid Dynamics

Numerical methods



We already detailed the fluid flow governing equations and clarified the methodology used to approximate them. **We now detail Boundary Conditions, BCs.**

In simulations, we need to decide BCs for every patch. These BCs approximate the interaction between the environment and the computational domain.



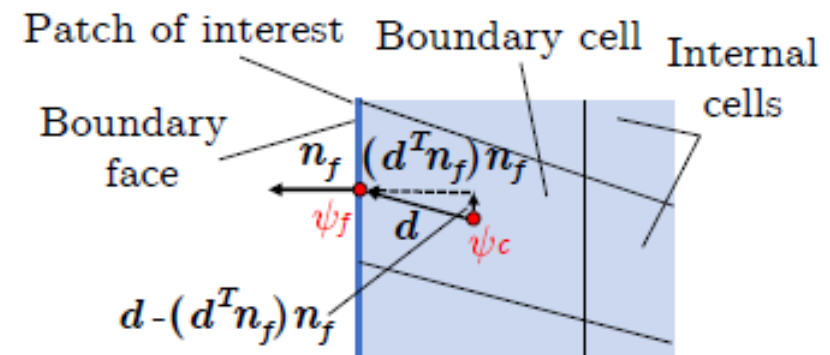
We start by introducing four basic types of BCs, from which all the others are deduced.

Fixed value

The value of the variable is fixed at the face.

When calculating advective terms the value at the face is **directly available** (instead of being interpolated from cell centres).

when calculating Laplacian terms



$$\psi_f - \psi_c \approx \nabla(\psi)_f d = \nabla(\psi)_f [(d^T n_f) n_f + (d - (d^T n_f) n_f)],$$

$$\nabla(\psi)_f n_f \approx \frac{\psi_f - \psi_c}{d^T n_f} - \frac{\nabla(\psi)_f (d - (d^T n_f) n_f)}{d^T n_f},$$

Fixed gradient

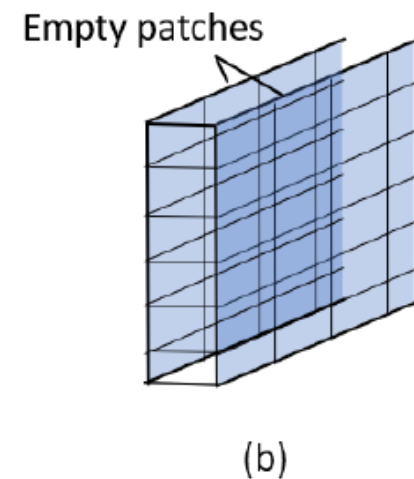
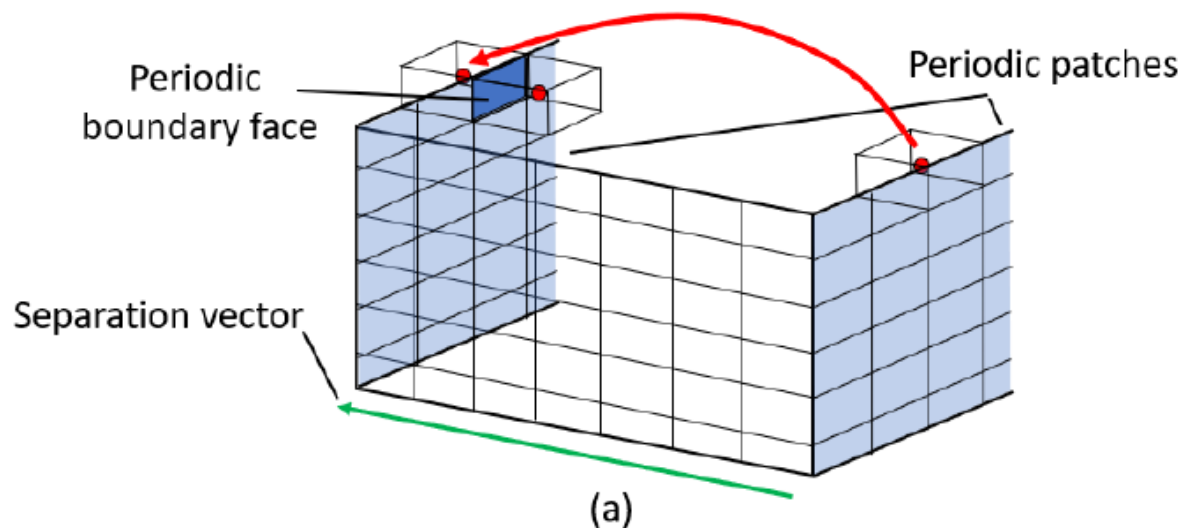
The BC actually fixes the normal gradient. When the normal gradient is set to be null, it is denoted as **zero gradient**.

Periodic

It enforces periodicity between two patches

Empty

Does nothing, the face is simply “jumped”, useful to mimic 2d cases.



Starting from basic types, we obtain specialized types.

Inlet

Variable	Basic type
Velocity	Fixed value
Pressure	Zero gradient
Turbulent quantities	Fixed value

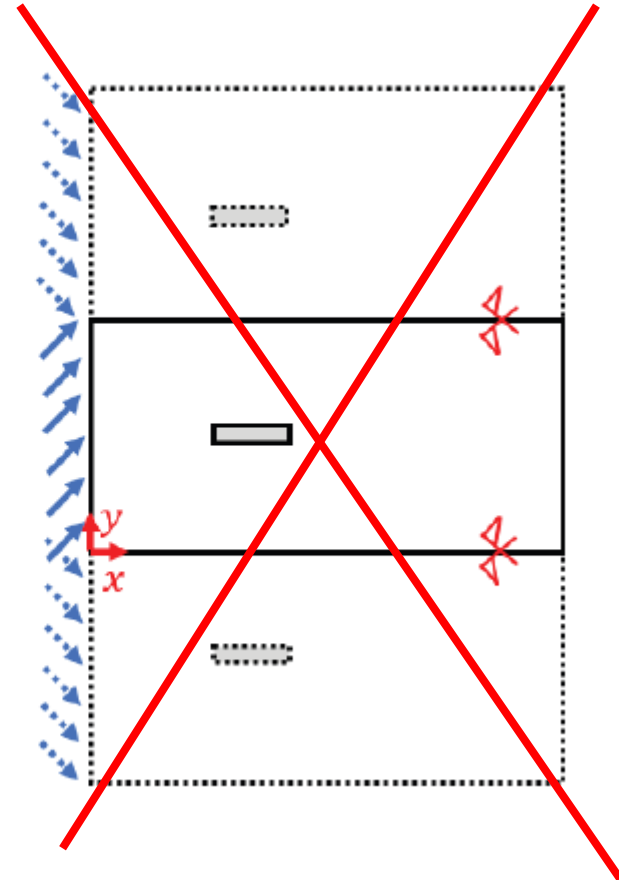
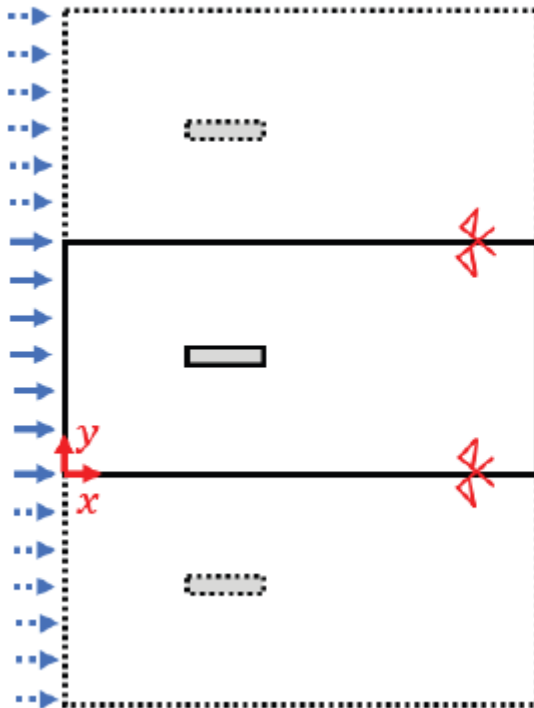
For LES, if incoming turbulence must be represented, it is necessary to use **Synthetic Inflow Generators**, able to provide appropriate unsteady velocity fields.

Outlet

Variable	Basic type
Velocity (for all components)	Zero gradient
Pressure	Fixed value (usually 0)
Turbulent quantities	Zero gradient

Symmetry

To be adopted whenever a symmetry exists (real or fictitious). Often used to simulate distant objects.



Slip

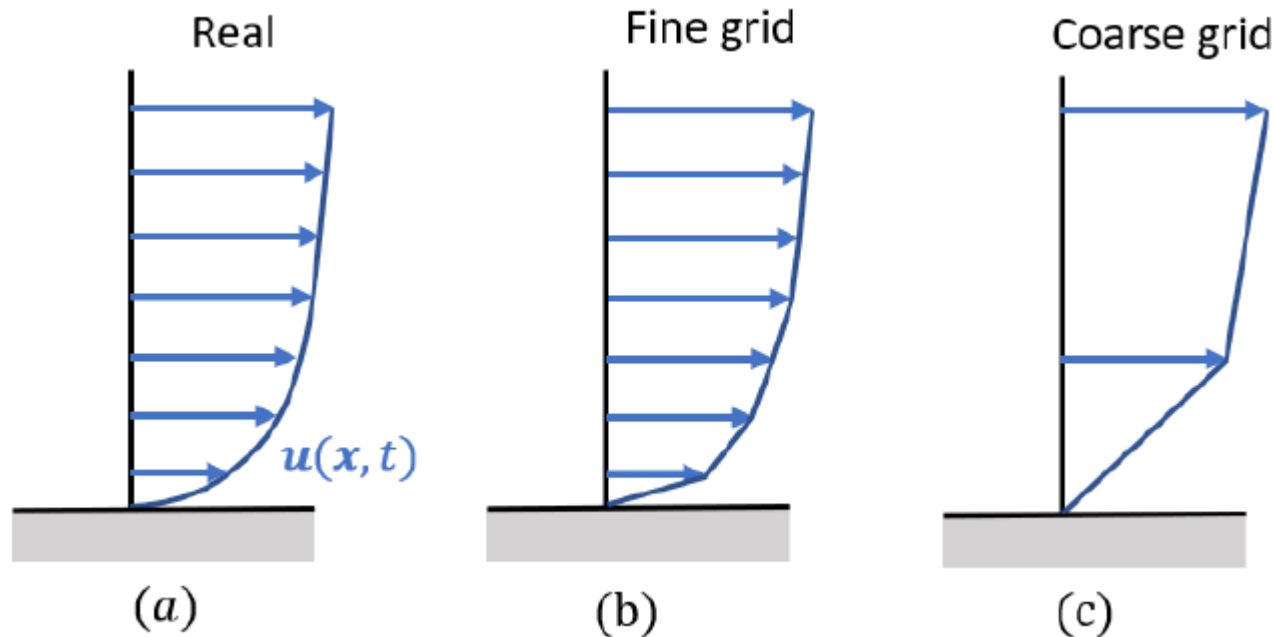
It fixes to zero the velocity normal to the path (impermeability), and adopts zero gradient for all other quantities. Useful to simulate walls without simulating the Boundary Layer.

No-slip

Similar to Slip but it also imposes a fixed zero value for the velocity component normal to the patch. **It might be argued that this represents a wall condition.**

However, wall require some additional special treatment...

The problem at walls lies in the fact that **boundary layers are usually very thin**, and it is difficult to mesh them accurately.

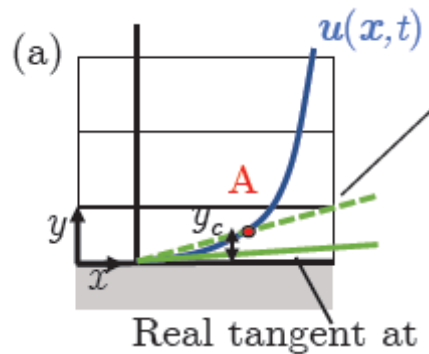


As it can be noticed, an insufficient resolution might lead to very inaccurate evaluations of the **gradients at the wall**, responsible for the wall friction.

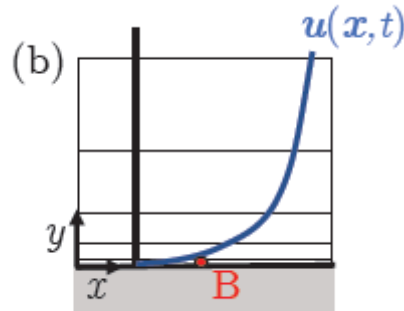
In particular, we might approximate the shear stress at the wall as

$$\tau_w = \mu \frac{u_c}{y_c}.$$

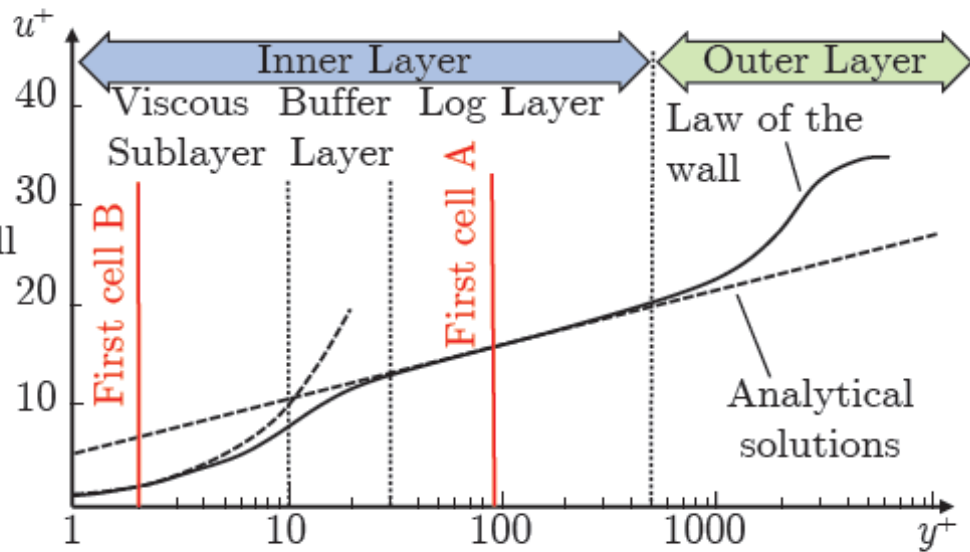
NO!



OK!



Secant between wall and first cell centre



(c)

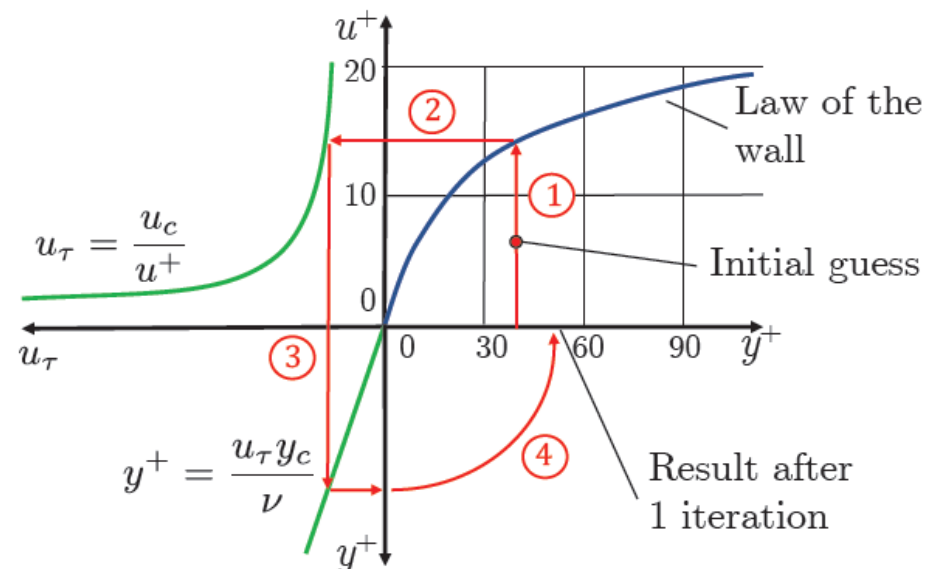
Essentially, if we do not want to mesh very finely at the wall, we know we will predict the shear stress inaccurately. To avoid it we use wall-functions.

We now present the problem in a rather idealized way. Assume we have an estimate of the wall shear stress. We calculate

$$u_\tau = \sqrt{\tau_w / \rho},$$

$$y_c^+ = y_c u_\tau / \nu,$$

We can proceed to an iterative correction of our initial guess following the scheme beside (which is just an example).



In the case of LES, the **length scale is usually damped at the wall** as

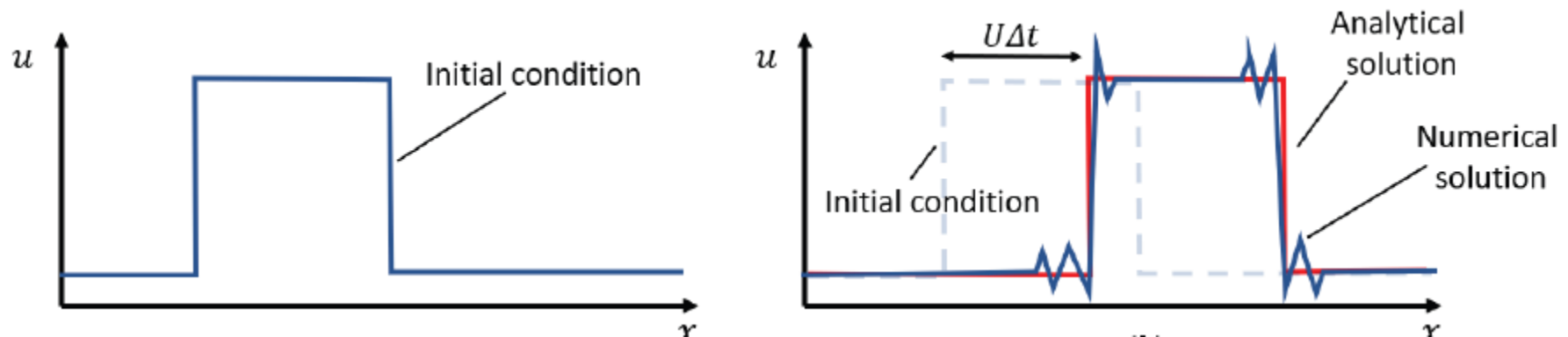
$$\Delta_{eff} = \min \left(\Delta, D(y^+) \frac{\kappa_K y}{C_s} \right), \quad \text{with} \quad D(y^+) = 1 - e^{-y^+/A^+},$$

where D is Van Driest damping function, κ_K is the von Kármán constant and $A^+ = 26$. In the proximity of the wall assume $\Delta \gg \kappa_K y / C_s$ ($D=1$), so that we have

$$\nu_{sgs} = (C_s \Delta_{eff})^2 \left| \frac{dU}{dy} \right| = \left(C_s \frac{\kappa y}{C_s} \right)^2 \left| \frac{dU}{dy} \right| = \kappa^2 y^2 \left| \frac{dU}{dy} \right|,$$

which is again Prandtl mixing length approach.

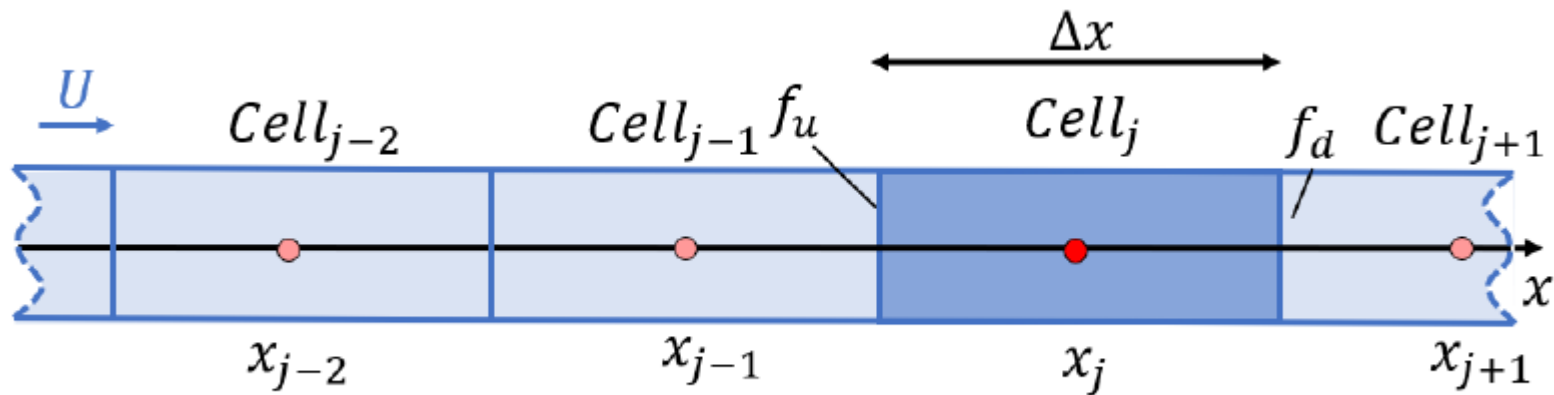
A common **numerical artefact** with deep implications on the stability of numerical simulations is the creation of wiggles, i.e. creation of new extremes.



It can be shown that their generation is linked to advective terms, especially if second order accurate schemes are used where sharp the solution is not smooth.

To solve the problem, we might try to selectively use first (upwind) or second order schemes, depending on the smoothness of the solution.

For the sake of exemplification, let us go back to our simple mono-dimensional example, adopting the FOU scheme

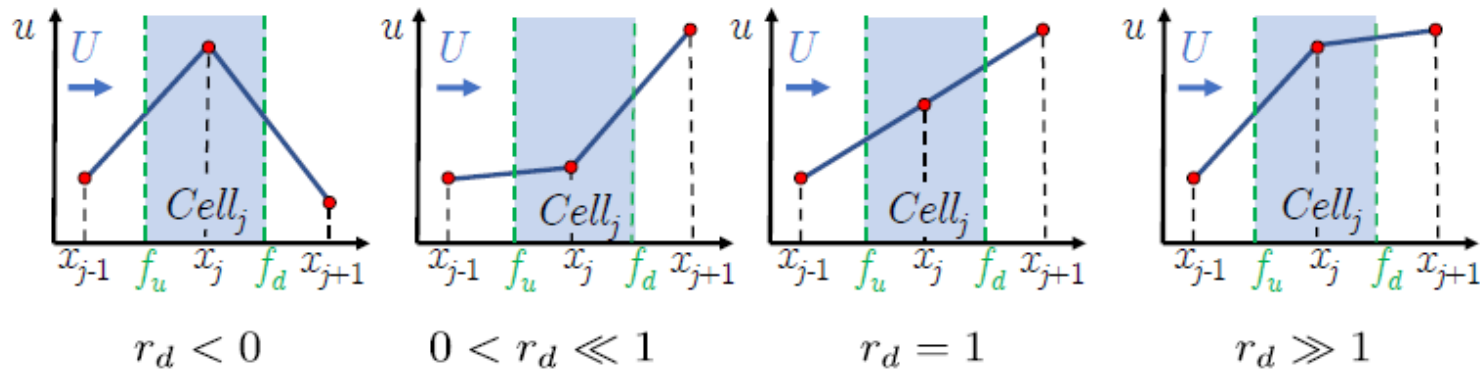


$$\frac{du_j}{dt} \Delta x = \text{flux}_{f_u} - \text{flux}_{f_d}. \quad \text{flux}_{f_u} = U u_{j-1}, \quad \text{flux}_{f_d} = U u_j,$$

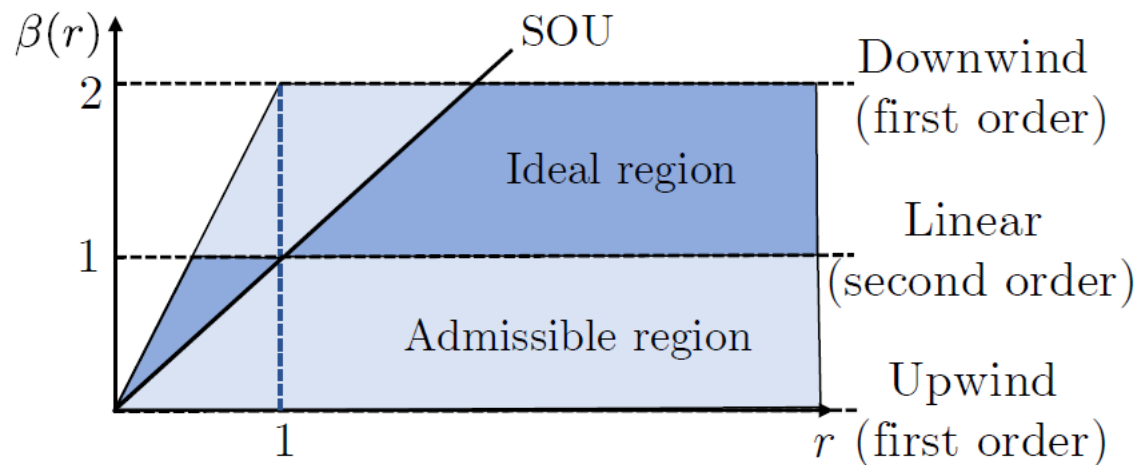
However, we might write a **blending between the FOU and the linear scheme** as

$$\begin{aligned} \text{flux}_{f_u} &= U \left\{ \beta(r_u) \left(\frac{u_{j-1} + u_j}{2} \right) + [1 - \beta(r_u)] u_{j-1} \right\} \\ \text{flux}_{f_d} &= U \left\{ \beta(r_d) \left(\frac{u_j + u_{j+1}}{2} \right) + [1 - \beta(r_d)] u_j \right\}, \end{aligned}$$

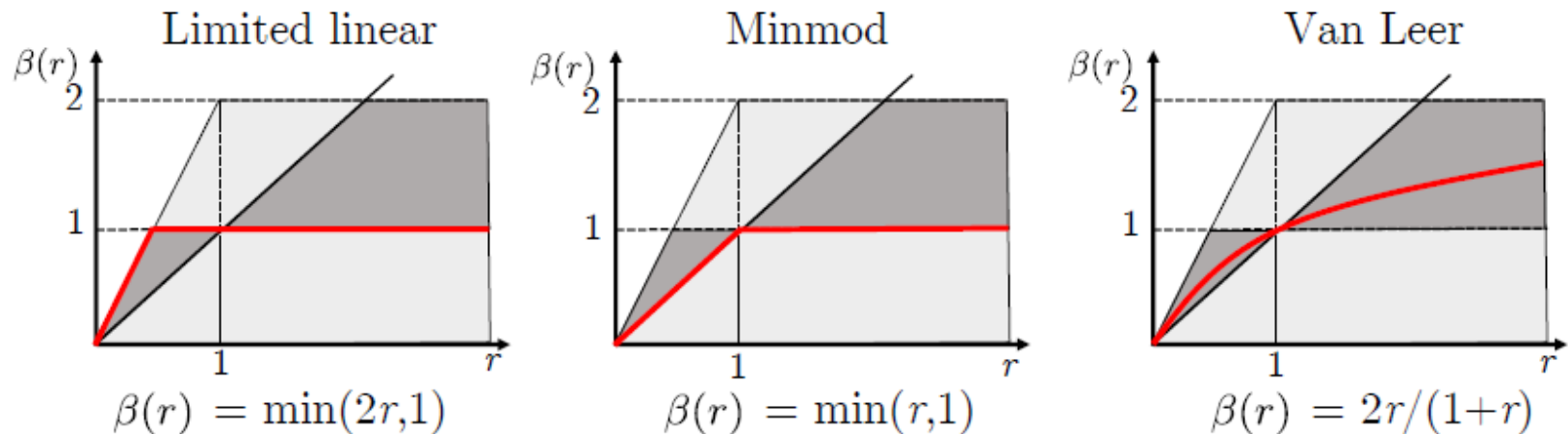
where r represents the ratio between two successive gradients



It can be shown that an appropriate selection of $\beta(r)$ leads to a number of schemes that we already encountered. Additionally, interesting schemes can be obtained by shaping $\beta(r)$



In particular, it can be shown that restricting the function $\beta(r)$ in some zones of the Swaby diagram, **the scheme is second order whenever possible and avoids the creation of new extrema.**



Schemes developed in such a way are known as Total Variation Diminishing Schemes, TVD schemes. Notice that, for this schemes, the discretization depends on the solution.

The overall conceptual organization of the solution strategy is as follow. Notice that this does not necessarily correspond to how things are implemented in all cases.

Not present in steady simulations

In transient simulations this is usually handled by pressure-velocity coupling algorithms

